Real-world (part 2) — run in parallel, fall back, and stream — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_patterns_practice.py`: the LCEL topic names three things every chain can do for free — run steps in parallel, fall back to a backup, and stream. This exercise uses each one once, on purpose, and proves it works. **If not:** Project 6's parallel lookups, fallbacks and streamed replies would be the first time you met `RunnableParallel`, `.with_fallbacks()` and `.stream()`.

Solution

```
# lcel_patterns_practice.py
import time

import openai
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable, RunnableLambda, RunnableParallel
from langchain_openai import ChatOpenAI

# why: fall back only on errors that pass on their own — a bug in your
# own code (a KeyError) must still fail loudly
TRANSIENT_ERRORS = (openai.APIConnectionError, openai.RateLimitError)
BACKUP_TEXT = "Sorry, the assistant is busy. Please try again soon."

def get_weather(inputs: dict) -> str:
    # why: the sleep stands in for a slow network call
    time.sleep(1)
    return "sunny in " + inputs["city"]

def get_news(inputs: dict) -> str:
    time.sleep(1)
    return "a new park opened in " + inputs["city"]

def build_lookup_step() -> Runnable:
    # how: RunnableParallel gives every function the same input, runs
    # them at the same time, and returns a dict with the same keys
    return RunnableParallel({"weather": get_weather, "news": get_news})
```

```
def backup_reply(inputs: dict) -> str:
    return BACKUP_TEXT

def with_backup(chain: Runnable) -> Runnable:
    # how: RunnableLambda turns a plain function into a step that
    # .with_fallbacks() accepts; the backup runs only on these errors
    return chain.with_fallbacks(
        [RunnableLambda(backup_reply)], exceptions_to_handle=TRANSIENT_ERRORS
    )

def stream_text(chain: Runnable, inputs: dict) -> str:
    # how: .stream() gives the reply in small pieces; print each one as
    # it arrives, and join them into the full text for the caller
    text = ""
    for chunk in chain.stream(inputs):
        print(chunk, end="", flush=True)
        text = text + chunk
    print()
    return text

def buggy_step(inputs: dict) -> str:
    return inputs["missing_key"]

def main() -> None:
    lookups = build_lookup_step()
    start = time.time()
    result = lookups.invoke({"city": "Lahore"})
    seconds = time.time() - start
    print(f"case 1 (keys)          -> {sorted(result.keys())}")
    print(f"case 1 (under 1.5 s?)  -> {seconds < 1.5}")

    prompt = ChatPromptTemplate.from_template(
        "Write one sentence about {city}."
    )
    # how: nothing listens on port 9 — a real, forced outage, no key used
    broken_model = ChatOpenAI(
        model="gpt-4o-mini", base_url="http://127.0.0.1:9", max_retries=0
    )
    broken_chain = with_backup(prompt | broken_model | StrOutputParser())
    reply = broken_chain.invoke({"city": "Lahore"})
    print(f"case 2 (outage)        -> backup used: {reply == BACKUP_TEXT}")

    bug_chain = with_backup(RunnableLambda(buggy_step))
    try:
        bug_chain.invoke({"city": "Lahore"})
        print("case 3 (bug)           -> hidden (wrong!)")
```

```
    except KeyError:
        print("case 3 (bug)           -> KeyError still raised")

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    print("case 4 (stream)        -> ", end="")
    live_chain = prompt | model | StrOutputParser()
    full_text = stream_text(live_chain, {"city": "Lahore"})
    print(f"case 4 (full text)     -> {len(full_text)} characters")

if __name__ == "__main__":
    main()
```

Expected output (the streamed sentence varies):

```
case 1 (keys)          -> ['news', 'weather']
case 1 (under 1.5 s?)  -> True
case 2 (outage)        -> backup used: True
case 3 (bug)           -> KeyError still raised
case 4 (stream)        -> Lahore is a busy, historic city in Pakistan.
case 4 (full text)     -> 44 characters
```